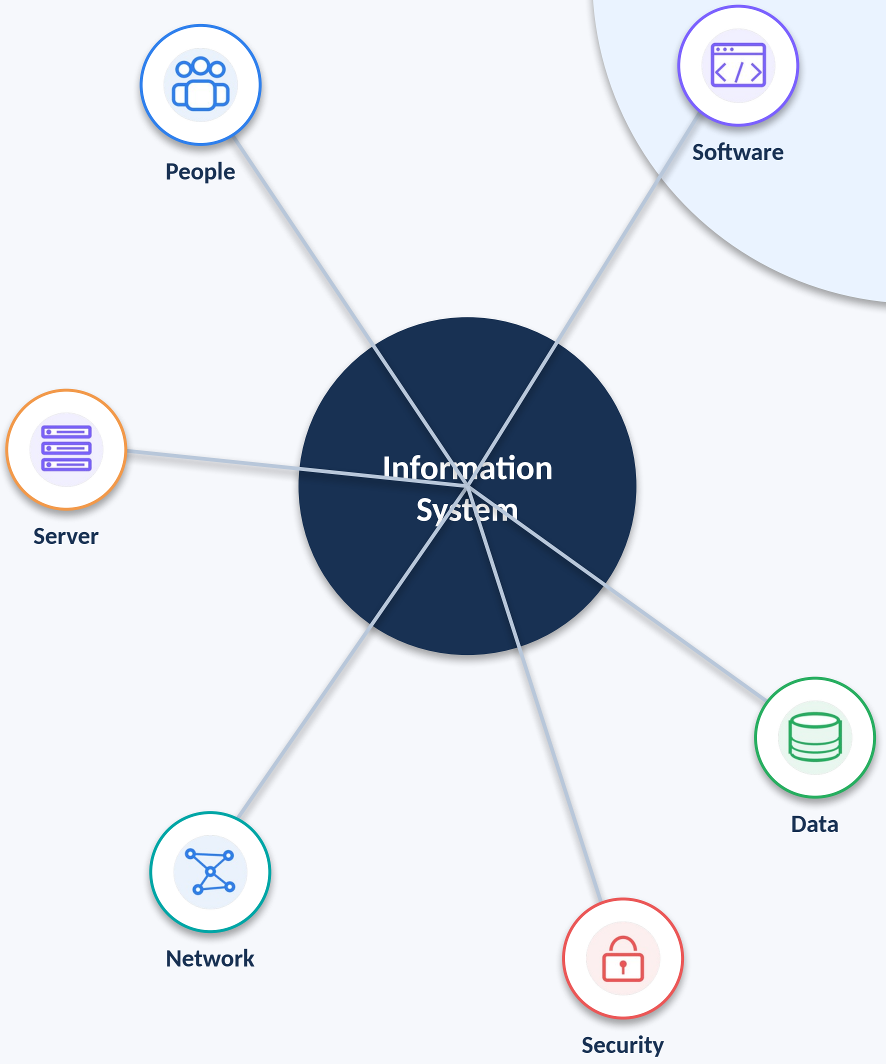
Information Systems

Components & Architecture

- **Big picture:** how people, data, software, hardware, and networks work together
- **Modern architecture:** client, frontend, network, backend, server, and database
- **Practical lens:** browser, UI, CLI, apps, HTTP/TCP, programming, and algorithms



MAP

Learning map: the large picture

A simple mental model for connecting the vocabulary.

Users / People — Need information → **Client Side** — UI, CLI, browser, apps → **Network** — HTTP, TCP/IP, APIs → **Server Side** — Backend processing → **Data Layer** — Databases, files, logs → **Outputs** — Reports and decisions

Input → Processing → Storage → Output → Feedback

Core cycle: data enters, is processed, stored, delivered, and improved through feedback.

COMPONENTS

Major components of an information system

These pieces work together; none is useful in isolation.

Hardware

Physical devices: computers, servers, routers

Software

Programs, operating systems, enterprise apps

Data

Raw facts organized into useful information

People

Users, managers, analysts, IT specialists

Processes

Rules, workflows, procedures, approvals

Networks

Internet, Wi-Fi, LAN, APIs, communication links

Security

Access control, encryption, backups, audit logs

Useful information emerges when components are coordinated.

ARCHITECTURE

Common architecture: layered system

A three-tier model plus network communication.

Presentation Layer / Frontend

UI, CLI, browser, mobile app, desktop app

Network Layer

HTTP/HTTPS, TCP/IP, APIs, internet/LAN/Wi-Fi

Application Layer / Backend

Business logic, programs, algorithms, authentication

Data Layer

Database, files, cloud storage, data warehouse, logs

Layer responsibilities

- **Frontend** → collects input + shows output
- **Network** → moves requests + responses
- **Backend** → applies rules + runs algorithms
- **Data** → stores facts + history

Most real systems use variations of this model.

CLIENT SIDE

Client side: where users interact

A client is any tool that sends requests and displays responses.

UI

Visual interface: buttons, forms, menus, dashboards

CLI

Text interface: commands, scripts, automation

Browser

Web client: Chrome, Edge, Safari, Firefox

Apps

Mobile or desktop software installed by users

Different clients can use the same backend and data layer.

CLIENT/SERVER

Server/client model: request and response

The client asks; the server processes; the client receives the result.

Client

- **Runs UI/CLI:** captures user actions
- **Sends requests:** asks server for work
- **Displays responses:** shows results

Request

HTTP/HTTPS

TCP/IP

Network communication

Response

Server

- **Receives requests:** from many clients
- **Runs backend:** logic + algorithms
- **Returns responses:** data, page, result

NETWORK

Communication: HTTP, TCP/IP, and APIs

Protocols define how systems exchange messages reliably and securely.

Protocol stack

HTTP / HTTPS User wants a page, data, login, or action

APIs Defined endpoints between apps and servers

TCP/IP Packets move across networks and arrive reliably

Example web exchange

Request: GET /products/42

Backend: validate -> compute -> query data

Response: product details + status code

HTTPS adds encryption and identity protection around HTTP.

BACKEND

Backend: programming, algorithms, and business logic

Backend code turns requests into trusted actions.

Receive request → Authenticate user → Run algorithm → Read/write data → Return response

Programming

Writing instructions that tell the system what to do.

Algorithm

Step-by-step method for solving a problem.

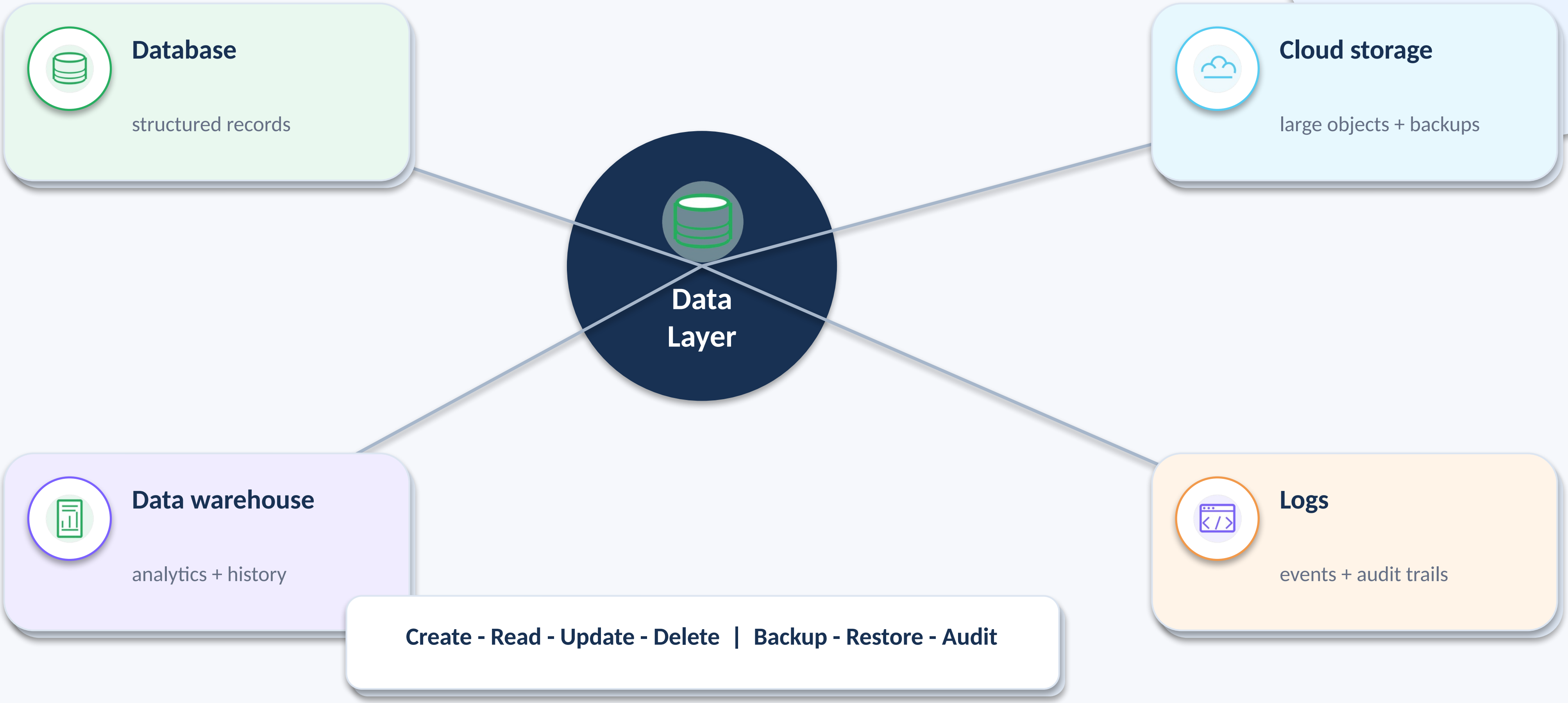
Business logic

Rules for prices, approvals, inventory, and workflows.

DATA

Data layer: memory of the system

Data stores keep facts, history, transactions, and evidence.



FRONTEND/BACKEND

Frontend and backend: what users see vs. what systems do

A useful distinction for designing, troubleshooting, and explaining systems.

Frontend

- **User-facing:** screens, forms, buttons, commands
- **Runs in:** browser, mobile app, desktop app, terminal
- **Main job:** collect input and present output

UI / CLI / Browser / Apps

request

response

Backend

- **System-facing:** servers, code, APIs, services
- **Runs on:** server infrastructure or cloud platform
- **Main job:** enforce rules and process data

Server / APIs / Algorithms / Database

EXAMPLE

Example: online shopping information system

Follow one customer action through the whole architecture.

Customer
chooses item

→ **App / Browser**
clicks Buy Now

→ **HTTPS request**
sent over TCP/IP

→ **Backend server**
processes order

→ **Database**
updates records

→ **Outputs**
receipt + report

Backend algorithm for Buy Now

1. Validate cart → 2. Check inventory → 3. Calculate price → 4. Process payment → 5. Save order → 6. Send confirmation

The same information system supports customer service, finance, inventory, and management reporting.

WRAP-UP

Summary: connect the terms

Use the architecture map to place each concept.

Client / Frontend
UI, CLI, browser, apps

Network
HTTP/HTTPS, TCP/IP, APIs

Server / Backend
programs, algorithms, business logic

Information System

Data Layer
database, files, cloud, logs

Information System
people + process + technology + data

Quick check
Can you trace a login, purchase, or report request from client to data and back?